

# รายงานโครงงาน

## Protonen

ระบบแสดงใบหน้าโปรโตเจนด้วยจอแสดงผล LED Matrix

ควบคุมผ่านเว็บบนไมโครคอนโทรลเลอร์ ESP32



ภาพที่ 1.1 ปก

จัดทำโดย

นายธนพล สุดสวาท

รหัสนักเรียน .....12186.....

โรงเรียน .....กาญจนาภิเษกวิทยาลัย นครปฐม...พระตำหนักสวนกุหลาบมัธยม.....

ปีการศึกษา ...2569.....

## บทคัดย่อ

โครงการนี้เป็นการออกแบบและสร้างระบบแสดงใบหน้าของตัวละครโปรโตเจน (Protogen) ซึ่งเป็นหน้ากากที่แสดงผลด้วยจอ LED Matrix ขนาด 8x8 จุด จำนวนหลายโมดูลต่อกัน โดยใช้ชิปควบคุม MAX7219 เป็นตัวขับจอ แบ่งใบหน้าออกเป็นสองฝั่งที่ใช้บอร์ดควบคุมแยกกัน ฝั่งขวาใช้ Arduino Nano ร่วมกับเซนเซอร์สัมผัส เพื่อเปลี่ยนรูปตาเมื่อมีการสัมผัส ส่วนฝั่งซ้ายใช้ไมโครคอนโทรลเลอร์ ESP32 Dev V1 ซึ่งสร้างเครือข่าย Wi-Fi แบบ Access Point และให้หน้าเว็บสำหรับควบคุม ผู้ใช้สามารถเลือกสีหน้า 8 แบบ (Normal, Happy, Angry, Sad, Wink, Surprised, Dead, UwU) ปรับความสว่าง ตั้งการกะพริบตา เปิดโหมดเปลี่ยนสีหน้าอัตโนมัติ วาดลวดลายเองบนจอ และเล่นเกมไดโนเสาร์กระโดดบนจอแสดงผลได้

ผลการทดลองแสดงให้เห็นว่าระบบสามารถแสดงผลและตอบสนองต่อคำสั่งได้ตามที่ออกแบบไว้

คำสำคัญ: protogen, LED Matrix, MAX7219, ESP32, Arduino, Web Server

## สารบัญ

|                                                |    |
|------------------------------------------------|----|
| บทคัดย่อ.....                                  | 2  |
| บทที่ 1 บทนำ.....                              | 5  |
| 1.1 ที่มาและความสำคัญ .....                    | 5  |
| 1.2 วัตถุประสงค์.....                          | 5  |
| 1.3 ขอบเขตของโครงการ .....                     | 6  |
| 1.4 ประโยชน์ที่คาดว่าจะได้รับ .....            | 6  |
| บทที่ 2 ทฤษฎีและเครื่องมือที่เกี่ยวข้อง .....  | 7  |
| 2.1 จอ LED Matrix 8x8 และชิป MAX7219 .....     | 7  |
| 2.2 ไมโครคอนโทรลเลอร์ Arduino .....            | 8  |
| 2.3 ไมโครคอนโทรลเลอร์ ESP32 .....              | 8  |
| 2.4 Frame Buffer .....                         | 8  |
| 2.5 เว็บเซิร์ฟเวอร์และ REST API แบบง่าย .....  | 8  |
| บทที่ 3 การออกแบบและขั้นตอนการดำเนินงาน .....  | 9  |
| 3.1 ภาพรวมของระบบ .....                        | 9  |
| 3.2 อุปกรณ์ที่ใช้.....                         | 9  |
| 3.3 การต่อวงจร.....                            | 10 |
| 3.4 การจัดเรียงแผงจอบนใบหน้า .....             | 11 |
| 3.5 ขั้นตอนการดำเนินงาน .....                  | 11 |
| 3.6 หลักการทำงานของโปรแกรมหลัก.....            | 15 |
| บทที่ 4 ผลการดำเนินงาน .....                   | 17 |
| 4.1 ผลการทำงานของฝั่งขวา (Arduino Nano).....   | 17 |
| 4.2 ผลการทำงานของฝั่งซ้าย (ESP32 Dev V1) ..... | 17 |
| 4.3 ผลการทดสอบฟังก์ชัน.....                    | 17 |
| 4.4 ปัญหาและแนวทางแก้ไข .....                  | 18 |
| บทที่ 5 สรุปผลและข้อเสนอแนะ .....              | 19 |

|                                          |    |
|------------------------------------------|----|
| 5.1 สรุปผล .....                         | 19 |
| 5.2 ข้อเสนอแนะและแนวทางการพัฒนาต่อ ..... | 19 |
| บรรณานุกรม .....                         | 20 |
| ภาคผนวก .....                            | 21 |
| ภาคผนวก ก ไฟล์ซอร์สโค้ด.....             | 21 |
| ภาคผนวก ข ภาพประกอบเพิ่มเติม.....        | 21 |

## บทที่ 1 บทนำ

### 1.1 ที่มาและความสำคัญ

#### PROTOGEN

เป็นแนวคิดตัวละครในวัฒนธรรมแฟนอาร์ตและงานคอสมเพลย์ที่มีใบหน้าเป็นจอแสดงผลอิเล็กทรอนิกส์ สามารถเปลี่ยนสีหน้าและแสดงอารมณ์ได้ การสร้างหน้ากากลักษณะนี้ต้องอาศัยความรู้หลายด้านรวมกัน ทั้งอิเล็กทรอนิกส์ การเขียนโปรแกรมสมองกลฝังตัว การสื่อสารไร้สาย และการออกแบบกราฟิกแบบพิกเซล จึงเป็นโครงการที่เหมาะสมกับการนำความรู้ในรายวิชามาประยุกต์ใช้จริง

ผู้จัดทำจึงพัฒนาระบบ Protogen ขึ้น โดยแบ่งใบหน้าเป็นสองฝั่งที่ใช้บอร์ดควบคุมแยกกัน ฝั่งซ้ายใช้ ESP32 Dev V1 ควบคุมผ่านเว็บเบราว์เซอร์ของโทรศัพท์มือถือ ส่วนฝั่งขวาใช้ Arduino Nano ร่วมกับเซนเซอร์สัมผัส เพื่อให้ผู้ใช้เปลี่ยนสีหน้าได้สะดวกโดยไม่ต้องแก้โค้ดหรือต่อสายเข้าคอมพิวเตอร์

อย่างไรก็ตามปัจจุบันทั้งสองฝั่งยังไม่ได้รวมเป็นบอร์ดเดียวกันเนื่องจากติดปัญหาเรื่องการจ่ายไฟที่ยังไม่สามารถแก้ไขได้ ประกอบกับข้อจำกัดด้านงบประมาณและความรู้ความสามารถของผู้จัดทำในปัจจุบัน จึงให้ทั้งสองฝั่งทำงานแยกอิสระต่อกัน

### 1.2 วัตถุประสงค์

1. เพื่อออกแบบและสร้างระบบแสดงใบหน้าโปรโตเจนด้วยจอ LED Matrix ชิป MAX7219
2. เพื่อพัฒนาโปรแกรมควบคุมการแสดงสีหน้า การกะพริบตา และความสว่าง
3. เพื่อสร้างหน้าเว็บควบคุมระบบผ่านเครือข่าย Wi-Fi ของ ESP32 โดยไม่ต้องพึ่งอินเทอร์เน็ต
4. เพื่อศึกษาการสื่อสารกับ MAX7219 และการจัดการหน่วยความจำภาพ (Frame Buffer)

### 1.3 ขอบเขตของโครงการ

- ฮาร์ดแวร์: จอ LED Matrix 8x8 ชิป MAX7219 ต่อแบบอนุกรม (Daisy Chain) แบ่งเป็นสองฝั่ง ฝั่งซ้ายควบคุมด้วย ESP32 Dev V1 (โปรแกรม proot2\_0) และฝั่งขวาควบคุมด้วย Arduino Nano (โปรแกรม theodor\_proot)
- แสดงผลได้ 8 สีหน้า ปรับความสว่างได้ 16 ระดับ (0–15) และกะพริบตาอัตโนมัติ
- ควบคุมผ่านเว็บเพจที่ ESP32 สร้างขึ้นเอง ไม่ต้องเชื่อมต่ออินเทอร์เน็ต
- มีโหมดวาดภาพเองและเกมไดโนเสาร์เป็นฟังก์ชันเสริม
- ไม่รวมการติดตามใบหน้าจริงของผู้สวมใส่ และไม่รวมการแสดงผลเสียง
- ทั้งสองฝั่งใช้บอร์ดแยกกันและทำงานอิสระต่อกัน ยังไม่ได้รวมเป็นบอร์ดเดียว เนื่องจากปัญหาการจ่ายไฟที่ยังไม่สามารถแก้ไขได้ในปัจจุบัน

### 1.4 ประโยชน์ที่คาดว่าจะได้รับ

- ได้ผลงานต้นแบบหน้ากากแสดงอารมณ์ที่ใช้งานได้จริง
- ได้ฝึกการเขียนโปรแกรมสมองกลฝังตัว การทำเว็บเซิร์ฟเวอร์บนไมโครคอนโทรลเลอร์ และการออกแบบ REST API ขนาดเล็ก
- เป็นฐานสำหรับต่อยอด เช่น เพิ่มสีหน้า ควบคุมด้วยเซนเซอร์ หรือเชื่อมต่อแอปมือถือ

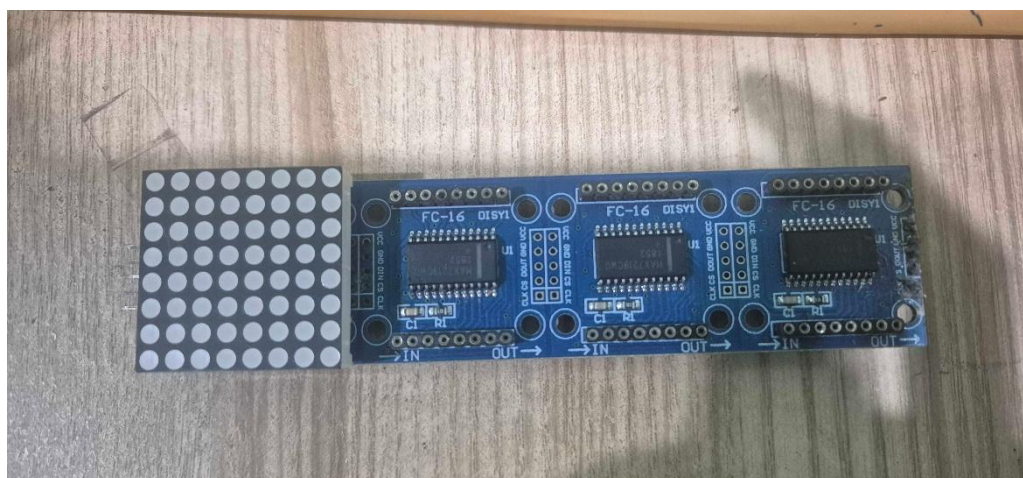
## บทที่ 2 ทฤษฎีและเครื่องมือที่เกี่ยวข้อง

### 2.1 จอ LED Matrix 8x8 และชิป MAX7219

จอ LED Matrix 8x8 ประกอบด้วยหลอด LED 64 ดวง จัดเรียงเป็นแถวและคอลัมน์ ชิปMAX7219 ทำหน้าที่สแกนขับ LED ทั้ง 64 ดวงแทนไมโครคอนโทรลเลอร์ รับคำสั่งผ่านสัญญาณแบบอนุกรม 3 เส้น ได้แก่ DIN (ข้อมูล), CLK (สัญญาณนาฬิกา) และ CS/LOAD (ส่งโหลดข้อมูล)นอกจากนี้ยังต่อโมดูลหลายตัวเป็นลูกโซ่ได้ โดยส่งข้อมูลต่อกันผ่านขา DOUT

การสื่อสารกับ MAX7219 ใช้คำสั่งขนาด 16 บิต ประกอบด้วยเลขรีจิสเตอร์ 8 บิตและค่าข้อมูล 8 บิต โดยรีจิสเตอร์ที่ใช้ในโครงงานนี้ ได้แก่

| รีจิสเตอร์    | ที่อยู่   | หน้าที่                                    |
|---------------|-----------|--------------------------------------------|
| DIGIT0-DIGIT7 | 0x01-0x08 | เก็บข้อมูลของแต่ละแถว (8 แถว)              |
| DECODE MODE   | 0x09      | ปิดการถอดรหัส (0x00) เพื่อควบคุมทีละพิกเซล |
| INTENSITY     | 0x0A      | ปรับความสว่าง 0-15                         |
| SCAN LIMIT    | 0x0B      | กำหนดจำนวนแถวที่สแกน (0x07 = ครบ 8 แถว)    |
| SHUTDOWN      | 0x0C      | เปิด/ปิดการแสดงผล                          |
| DISPLAY TEST  | 0x0F      | โหมดทดสอบไฟติดทั้งหมด                      |



ภาพที่ 2.1 โมดูล LED Matrix 8x8 ชิป MAX7219

## 2.2 ไมโครคอนโทรลเลอร์ Arduino

Arduino เป็นบอร์ดไมโครคอนโทรลเลอร์ที่เขียนโปรแกรมด้วยภาษา C/C++ ในโครงงานนี้ใช้ Arduino Nano ควบคุมใบหน้าผิงขวา ร่วมกับไลบรารี MaxMatrix ซึ่งช่วยจัดการการส่งข้อมูลไปยัง MAX7219 หลายตัว และใช้เซนเซอร์สัมผัสต่อที่ขา 2 เป็นอินพุต

## 2.3 ไมโครคอนโทรลเลอร์ ESP32

ESP32 เป็นไมโครคอนโทรลเลอร์ที่มี Wi-Fi และ Bluetooth ในตัว ทำงานเร็วและมีหน่วยความจำมากกว่า Arduino รุ่นพื้นฐาน เหมาะสำหรับสร้างเว็บเซิร์ฟเวอร์ขนาดเล็ก ในโครงงานนี้ใช้ ESP32 Dev V1 ควบคุมใบหน้าผิงซ้าย โดยใช้ไลบรารี WiFi.h สำหรับสร้างจุดเชื่อมต่อ (Access Point) ไลบรารี WebServer.h สำหรับรับคำสั่ง HTTP และไลบรารี Preferences.h สำหรับบันทึกค่าลงหน่วยความจำถาวร (ใช้เก็บคะแนนสูงสุดของเกม)

## 2.4 Frame Buffer

Frame Buffer คือพื้นที่หน่วยความจำที่เก็บภาพทั้งหมดก่อนส่งไปแสดงบนจอ ในโครงงานนี้ใช้อาร์เรย์ framebuf[7][8] คือ 7 แถว แถวละ 8 ไบต์ (1 ไบต์ต่อ 1 แถว 8 พิกเซล) การวาดภาพทำโดยเขียนลง Frame Buffer ก่อน แล้วเรียก fbFlush() เพื่อส่งทุกแถวออกไปพร้อมกัน ทำให้ภาพไม่กะพริบหรือขาดระหว่างการอัปเดต

## 2.5 เว็บเซิร์ฟเวอร์และ REST API แบบง่าย

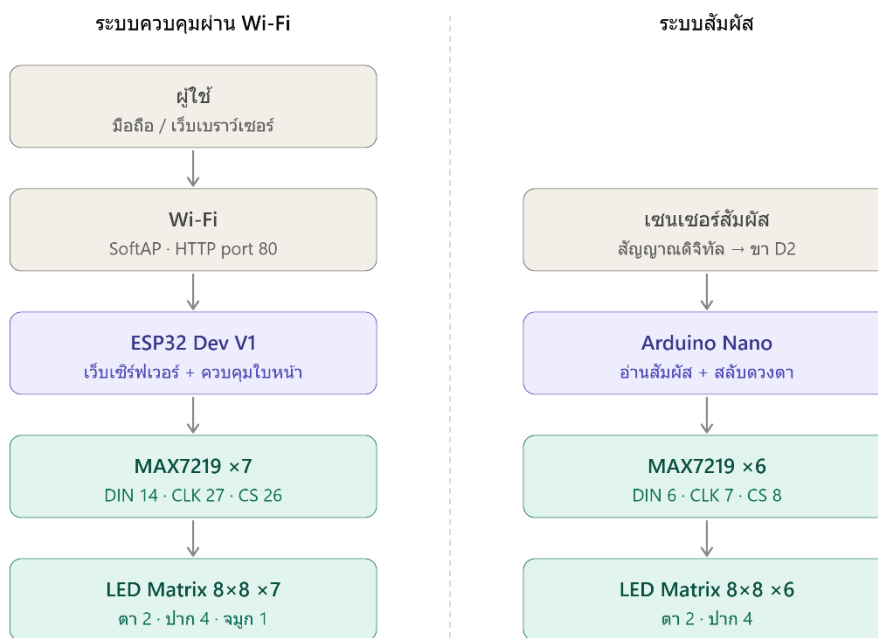
ESP32 ทำหน้าที่เป็นเว็บเซิร์ฟเวอร์ที่พอร์ต 80 โดยเมื่อเข้าที่ที่อยู่ 192.168.4.1 จะได้น้ำเว็บควบคุม และหน้าเว็บจะส่งคำสั่งไปยังเส้นทาง /api/... ด้วยวิธี HTTP GET เพื่อสั่งเปลี่ยนสีหน้า ปรับค่า และควบคุมเกม

## บทที่ 3 การออกแบบและขั้นตอนการดำเนินงาน

### 3.1 ภาพรวมของระบบ

ใบหน้าแบ่งเป็นสองฝั่งซ้าย-ขวา แต่ละฝั่งใช้บอร์ดควบคุมแยกกัน ฝั่งซ้ายควบคุมด้วย ESP32 Dev V1 (โปรแกรม proot2\_0) ฝั่งขวาควบคุมด้วย Arduino Nano (โปรแกรม theodor\_proot) โดยระบบฝั่งซ้ายประกอบด้วยส่วนหลัก 3 ส่วน ได้แก่ (1) ส่วนแสดงผล คือ LED Matrix ต่อเป็นลูกโซ่ (2) ส่วนควบคุม คือ ESP32 ที่ประมวลผลและสร้างภาพ (3) ส่วนผู้ใช้ คือ โทรศัพท์หรือคอมพิวเตอร์ที่เปิดหน้าเว็บผ่าน Wi-Fi ส่วนฝั่งขวาประกอบด้วยจอ LED Matrix, Arduino Nano และเซนเซอร์สัมผัส ทำงานตามโปรแกรมที่เขียนไว้โดยไม่ผ่าน Wi-Fi

สาเหตุที่ไม่ได้รวมสองฝั่งเป็นบอร์ดเดียวกันคือติดปัญหาเรื่องการจ่ายไฟที่ยังไม่สามารถแก้ไขได้ในปัจจุบัน เนื่องจากข้อจำกัดด้านงบประมาณและความรู้ความสามารถของผู้จัดทำ จึงใช้วิธีแยกบอร์ดและแยกการจ่ายไฟให้แต่ละฝั่งไปก่อน



ภาพที่ 3.1 แผนภาพบล็อกของระบบ (ฝั่งซ้าย: ผู้ใช้ → Wi-Fi → ESP32 Dev V1 → MAX7219 → LED Matrix / ฝั่งขวา: เซนเซอร์สัมผัส → Arduino Nano → MAX7219 → LED Matrix)

### 3.2 อุปกรณ์ที่ใช้

| ที่ | รายการ                         | จำนวน | หมายเหตุ                                                         |
|-----|--------------------------------|-------|------------------------------------------------------------------|
| 1   | ESP32 Dev V1 (ESP32 DevKit V1) | 1     | ควบคุมใบหน้าฝั่งซ้าย (proot2)                                    |
| 2   | โมดูล LED Matrix 8x8 MAX7219   | 13    | ฝั่งซ้าย 7 ตัว (ตา 2, จมูก 1, ปาก 4) ฝั่งขวา 6 ตัว (ตา 2, ปาก 4) |
| 3   | Arduino Nano                   | 1     | ควบคุมใบหน้าฝั่งขวา (theodor_proot) ใช้โมดูล 6 ตัว               |
| 4   | เซนเซอร์สัมผัส (Touch Sensor)  | 1     | ฝั่งขวา                                                          |
| 5   | สายจัมเปอร์ / แหล่งจ่ายไฟ 5V   | -     | แยกแหล่งจ่ายไฟแต่ละฝั่ง [ระบุแบตเตอรี่/พาวเวอร์แบงก์]            |
| 6   | หน้ากากและโครงยึดจอ            | 1     | ช่วงแรก: โฟม EVA และไม้ไผ่ / ช่วงหลัง: แผ่น Plaswood             |

### 3.3 การต่อวงจร

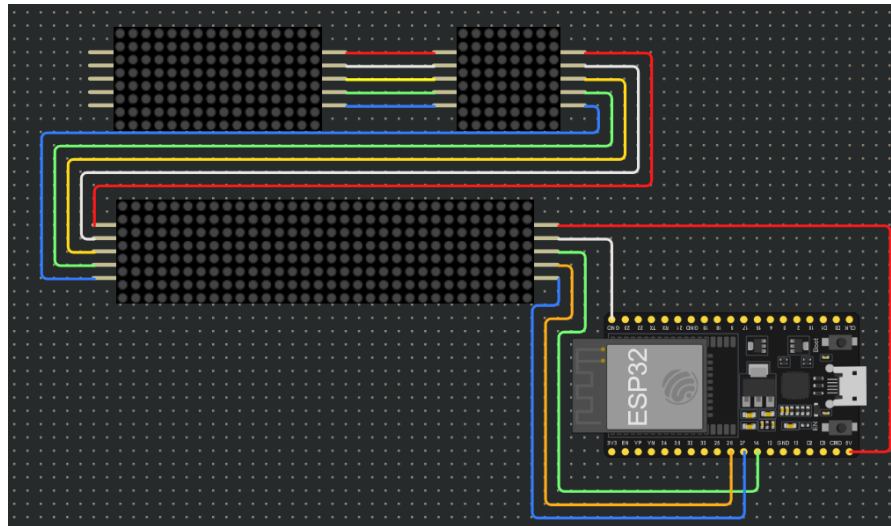
#### 3.3.1 ฝั่งขวา (Arduino Nano)

| ขาของ LED Matrix | ขา Arduino | หมายเหตุ      |
|------------------|------------|---------------|
| DIN              | 6          | ข้อมูล        |
| CLK              | 7          | สัญญาณนาฬิกา  |
| CS               | 8          | โหนดข้อมูล    |
| Touch Sensor     | 2          | อินพุตดิจิทัล |
| VCC / GND        | 5V / GND   | ไฟเลี้ยง      |

#### 3.3.2 ฝั่งซ้าย (ESP32 Dev V1)

| ขาของ LED Matrix | ขา ESP32 | หมายเหตุ     |
|------------------|----------|--------------|
| DIN              | GPIO 14  | ข้อมูล       |
| CLK              | GPIO 27  | สัญญาณนาฬิกา |
| CS               | GPIO 26  | โหนดข้อมูล   |

| ขาของ LED Matrix | ขา ESP32 | หมายเหตุ |
|------------------|----------|----------|
| VCC / GND        | 5V / GND | ไฟเลี้ยง |



ภาพที่ 3.3 แผนผังการต่อวงจร (Wiring Diagram) ระหว่าง ESP32 และ LED Matrix

### 3.4 การจัดเรียงแผงจอบนใบหน้า

โมดูลฝั่งซ้ายทั้ง 7 ตัวต่อเป็นลูกโซ่ตามลำดับทางกายภาพ ดังตารางต่อไปนี้ (ส่วนฝั่งขวาใช้โมดูล 6 ตัว

คือ ตา 2 ตัวและปาก 4 ตัว ไม่ได้ใช้โมดูลจมูก)

| ลำดับในโซ่ | แผง          | ตำแหน่งบนใบหน้า   |
|------------|--------------|-------------------|
| 0          | PANEL_EYE0   | ดวงตา (ข้างหนึ่ง) |
| 1          | PANEL_EYE1   | ดวงตา (อีกข้าง)   |
| 2          | PANEL_MOUTH3 | ปาก ส่วนที่ 4     |
| 3          | PANEL_MOUTH2 | ปาก ส่วนที่ 3     |
| 4          | PANEL_MOUTH1 | ปาก ส่วนที่ 2     |
| 5          | PANEL_MOUTH0 | ปาก ส่วนที่ 1     |
| 6          | PANEL_NOSE   | จมูก              |

### 3.5 ขั้นตอนการดำเนินงาน

## ขั้นตอนที่ 1 ศึกษาและวางแผน

ศึกษาหลักการทำงานของ MAX7219 และไลบรารีที่เกี่ยวข้อง กำหนดจำนวนจอ ตำแหน่งบนใบหน้า และสีหน้าที่ต้องการแสดง

## ขั้นตอนที่ 2 ออกแบบลวดลายพิกเซล

ออกแบบรูปตา จมูก และปากบนตาราง 8x8 จุด จากนั้นแปลงเป็นรหัสไบนารี 8 บิตต่อแถว เช่น 0b11100000 หมายถึงเปิด LED 3 ดวงแรกของแถวนั้น โดยปากหนึ่งชุดต้องแบ่งออกเป็น 4 แถวต่อกัน

## ขั้นตอนที่ 3 ออกแบบและสร้างโครงหน้ากาก

โครงหน้ากากทำหน้าที่ยึดจอ LED Matrix บอร์ดควบคุม และสายไฟต่างๆ ให้อยู่ในตำแหน่งที่ถูกต้องในการทำงานมีการเปลี่ยนวัสดุสองช่วง ดังนี้

- ช่วงแรก: ใช้โฟม EVA เป็นฐานของหน้ากาก และใช้ไม้ไผ่เป็นโครงเสริมเพื่อรองรับโครงสร้าง (ดูภาพที่ ข.1 ในภาคผนวก ข)
- ช่วงต่อมา: เปลี่ยนมาใช้แผ่น Plaswood เนื่องจากมีความแข็งแรงมากกว่า

ระหว่างการทำงานมีการปรับปรุงแบบการออกแบบหน้ากากหลายครั้ง เช่น

ปรับความยาวของส่วนหน้าและย้ายตำแหน่งจอ LED Matrix ดังแสดงในภาพที่ ข.2-ข.4 ในภาคผนวก ข

## ขั้นตอนที่ 4 ประกอบวงจรและเขียนโปรแกรมฝังขวด้วย Arduino Nano

ต่อโมดูลจอ 6 ตัวเข้ากับ Arduino Nano เขียนโปรแกรม (theodor\_proot.ino) ใช้ไลบรารี MaxMatrix ฟังก์ชัน drawBitmap() วาดภาพลงโมดูลที่กำหนด ในรูปหลักจะแสดงรูปปากตลอด และตรวจสอบเซนเซอร์สัมผัส ถ้าถูกสัมผัสจะเปลี่ยนตาเป็นรูปอีกแบบ ถ้าไม่ถูกสัมผัสจะแสดงตาปกติ รอ 3 วินาที แล้วแสดงภาพตากะพริบ 250 มิลลิวินาที

## ขั้นตอนที่ 5 พัฒนาฝังซ้าย (ESP32) : ส่วนจับจอ

เขียนฟังก์ชันสื่อสารกับ MAX7219 เองโดยไม่ใช้ไลบรารีภายนอก ได้แก่ spiSendByte()

ส่งข้อมูลที่ละบิตผ่าน DIN และ CLK, maxSendAll() ส่งคำสั่งเดียวกันให้ทุกชิป, maxSendRow()

ส่งข้อมูลแถวเดียวให้ทุกชิปพร้อมกัน, maxInit() ตั้งค่าเริ่มต้น และ fbFlush() ส่ง Frame Buffer

ทั้งหมดออกจอ

#### ขั้นตอนที่ 6 พัฒนาฝั่งซ้าย (ESP32) : สีหน้าและการกะพริบตา

เก็บสีหน้าทั้ง 8 แบบไว้ในโครงสร้าง Expr (ชื่อ, ตา 2 แฉง, ปาก 4 แฉง) ฟังก์ชัน drawExpression()

นำข้อมูลของสีหน้าที่เลือกไปใส่ Frame Buffer แล้วส่งออกจอ การกะพริบตาใช้ updateBlinkAnim()

ซึ่งค่อยๆ ปิดตาจากบนลงล่างทีละแถว (เรียกซ้ำทุก 16 มิลลิวินาที) คงค้างไว้ 55 มิลลิวินาที แล้วเปิดกลับ

โดยไม่ใช้ delay() เพื่อให้เว็บเซิร์ฟเวอร์ยังตอบสนองได้ระหว่างกะพริบ

| ที่ | สีหน้า    | ลักษณะ                   |
|-----|-----------|--------------------------|
| 1   | Normal    | ตาและปากปกติ             |
| 2   | Happy     | ตาโค้งยิ้ม               |
| 3   | Angry     | ตาเฉียงลง ปากเป็นเส้นตรง |
| 4   | Sad       | ตาดกกลง                  |
| 5   | Wink      | ตาข้างหนึ่งปิดเป็นเส้น   |
| 6   | Surprised | ตากกลมโต ปากอ้า          |
| 7   | Dead      | ตาเป็นรูปกากบาท          |
| 8   | UwU       | ตารูปเพชร ปากยิ้มแบบแมว  |



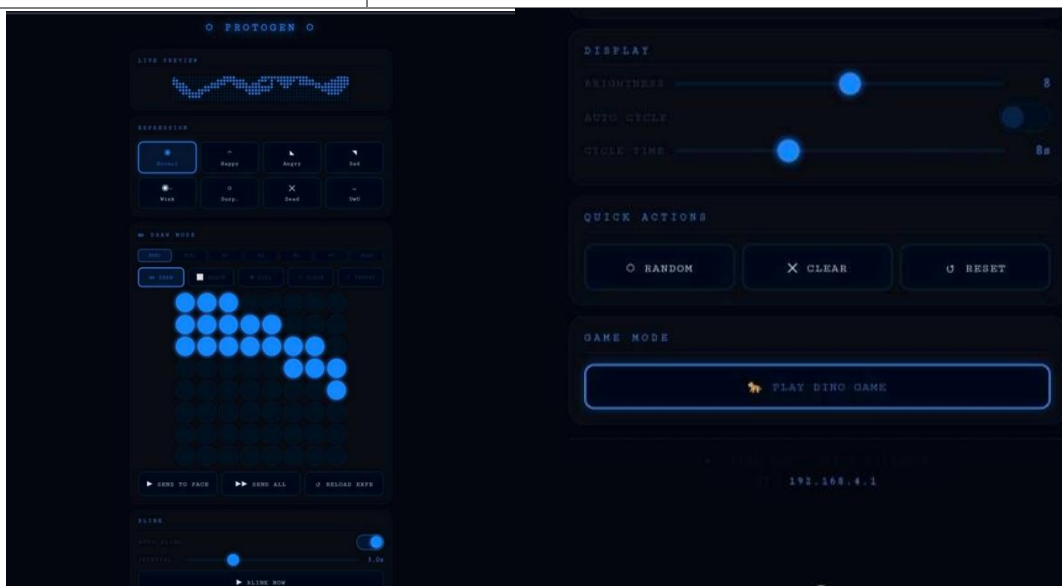
ภาพที่ 3.6 สีหน้าทั้ง 8 แบบที่แสดงบนจอ

### ขั้นตอนที่ 7 พัฒนาฝั่งซ้าย (ESP32) : Wi-Fi และเว็บเซิร์ฟเวอร์

ในฟังก์ชัน `setup()` ESP32 เปิด Access Point ชื่อ "Theodor\_de" ด้วย `WiFi.softAP()` แล้วเริ่มเว็บเซิร์ฟเวอร์ ผู้ใช้เชื่อมต่อ Wi-Fi ดังกล่าวและเปิดเบราว์เซอร์ไปที่ `http://192.168.4.1` จะพบหน้าควบคุม ซึ่งฝังเป็นข้อความ HTML/CSS/JavaScript อยู่ในโค้ด (`HTML_PAGE`) ส่วนฟังก์ชัน `loop()` เรียก `server.handleClient()` ตลอดเวลาเพื่อรับคำสั่ง

เส้นทาง API ที่สร้างขึ้นมีดังนี้

| เส้นทาง                                            | หน้าที่                                                |
|----------------------------------------------------|--------------------------------------------------------|
| <code>/api/expr</code>                             | เปลี่ยนสีหน้า                                          |
| <code>/api/blink</code>                            | เปิด/ปิดและตั้งช่วงเวลากะพริบตา                        |
| <code>/api/brightness</code>                       | ปรับความสว่าง                                          |
| <code>/api/auto</code>                             | เปิด/ปิดโหมดเปลี่ยนสีหน้าอัตโนมัติ                     |
| <code>/api/draw</code> , <code>/api/drawAll</code> | วาดภาพเองทีละแผงหรือทั้งหมด (ส่งข้อมูลเป็นเลขฐานสิบหก) |
| <code>/api/clear</code> , <code>/api/reset</code>  | ล้างจอ / กลับสู่สีหน้าเริ่มต้น                         |
| <code>/api/status</code>                           | ส่งสถานะปัจจุบันเป็น JSON                              |
| <code>/api/game/*</code>                           | ควบคุมเกม (start, jump, duck, pause, exit, status)     |



### ภาพที่ 3.7 หน้าเว็บควบคุม Protogen Face บนโทรศัพท์มือถือ (Screenshot)

#### ขั้นตอนที่ 8 พัฒนาฟังก์ชันเสริม

- โหมดวาดภาพ (Draw Mode): ผู้ใช้คลิกพิกเซลบนหน้าเว็บ แล้วส่งข้อมูลไปแสดงบนจอจริง
- Live Preview: หน้าเว็บมีตัวอย่างใบหน้าแบบสดบน canvas ให้เห็นก่อนส่งออกจอ
- โหมดอัตโนมัติ: เปลี่ยนสีหน้าวนไปทุกช่วงเวลาที่กำหนด (ค่าเริ่มต้น 8 วินาที)
- เกมไดโนเสาร์: ใช้พื้นที่จอ 32x8 พิกเซล มี 3 ระดับความยาก มีสิ่งกีดขวาง ตรวจจับการชน นับคะแนน และบันทึกคะแนนสูงสุดลง Preferences
- แอนิเมชันเปิดเครื่อง (Boot Animation): ไฟสว่างกวาดทีละแถวแล้วดับ ก่อนแสดงสีหน้าแรก

#### ขั้นตอนที่ 9 อัปโหลดโปรแกรมและทดสอบ

1. ติดตั้ง Arduino IDE และเพิ่มบอร์ด ESP32 ผ่าน Boards Manager
2. เปิดไฟล์โปรแกรม เลือกบอร์ดและพอร์ต COM ให้ถูกต้อง
3. กด Upload แล้วเปิด Serial Monitor ที่ 115200 baud เพื่อดูข้อความบูต
4. ใช้โทรศัพท์เชื่อมต่อ Wi-Fi ที่ ESP32 สร้าง แล้วเปิดเบราว์เซอร์ไปที่ 192.168.4.1
5. ทดสอบทุกปุ่มบนหน้าเว็บ บันทึกผล และแก้ไขข้อบกพร่อง

### 3.6 หลักการทำงานของโปรแกรมหลัก

สำหรับฝั่งซ้าย เมื่อจ่ายไฟ ESP32 จะตั้งค่าขาสัญญาณ เริ่มต้น MAX7219 ทั้ง 7 ตัว

เล่นแอนิเมชันเปิดเครื่อง แสดงสีหน้าเริ่มต้น เปิด Wi-Fi และเว็บเซิร์ฟเวอร์

จากนั้นเข้าสู่ลูปหลักซึ่งทำงานเรียงลำดับ คือ รับคำสั่งจากเว็บ

ถ้าเกมกำลังเล่นอยู่ให้ประมวลผลเกมก่อนแล้วจบรอบ

ถ้าไม่ใช่ให้ตรวจเวลากะพริบตาและเวลาเปลี่ยนสีหน้าอัตโนมัติ โดยใช้ millis() จับเวลาแทน delay()

เพื่อไม่ให้โปรแกรมค้าง

สำหรับฝั่งขวา เมื่อจ่ายไฟ Arduino Nano จะเริ่มต้นไลบรารี MaxMatrix ตั้งความสว่างและล้างจอ จากนั้นลูปหลักจะอ่านค่าจากเซนเซอร์สัมผัสที่ขา 2 และวาดรูปปากบนโมดูลที่ 2-5 ตลอดเวลา ถ้าเซนเซอร์ถูกสัมผัสจะเปลี่ยนตาเป็นอีกรูปแบบ ถ้าไม่ถูกสัมผัสจะแสดงตาปกติ รอ 3 วินาที แล้วแสดงภาพตากะพริบ 250 มิลลิวินาที ทั้งนี้เนื่องจากโปรแกรมใช้ delay() ระหว่างรอ การตอบสนองต่อการสัมผัสจึงอาจล่าช้าได้

## บทที่ 4 ผลการดำเนินงาน

### 4.1 ผลการทำงานของฝั่งขวา (Arduino Nano)



ภาพที่ 4.1 ผลการทำงานของฝั่งขวา (Arduino Nano)

### 4.2 ผลการทำงานของฝั่งซ้าย (ESP32 Dev V1)

เชื่อมต่อ WIFI ได้เกือบจะทันที ระยะเวลาใช้งานจากตัวควบคุม สูงสุดถึง 30เมตร



ภาพที่ 4.2 เกมกระโดดข้ามบล็อก(ซ้าย) หน้า DEAD(ขวา)

### 4.3 ผลการทดสอบฟังก์ชัน

| ที่ | รายการทดสอบ             | ผลที่คาดหวัง           | ผลที่ได้       |
|-----|-------------------------|------------------------|----------------|
| 1   | เปลี่ยนสีหน้าทั้ง 8 แบบ | จอเปลี่ยนตามที่เลือก   | [ผ่าน/ไม่ผ่าน] |
| 2   | ปรับความสว่าง 0-15      | ความสว่างเปลี่ยนตามค่า | ผ่าน           |

| ที่ | รายการทดสอบ                | ผลที่คาดหวัง               | ผลที่ได้ |
|-----|----------------------------|----------------------------|----------|
| 3   | กะพริบตาอัตโนมัติ          | ตากะพริบตามช่วงเวลาที่ตั้ง | ผ่าน     |
| 4   | โหมดเปลี่ยนสีหน้าอัตโนมัติ | สีหน้าเปลี่ยนวนทุกช่วงเวลา | ผ่าน     |
| 5   | วาดภาพเอง                  | ภาพบนเว็บตรงกับจอจริง      | ผ่าน     |
| 6   | เกมไดโนเสาร์               | กระโดด/หมอบ/นับคะแนนได้    | ผ่าน     |
| 7   | บันทึกคะแนนสูงสุด          | ค่าไม่หายหลังรีสตาร์ท      | ผ่าน     |

#### 4.4 ปัญหาและแนวทางแก้ไข

| ที่ | ปัญหาที่พบ                                                                                                             | การแก้ไข                                                                                                                                    |
|-----|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | ไม่สามารถรวมฝั่งซ้าย (ESP32 Dev V1) และฝั่งขวา (Arduino Nano) เป็นบอร์ดเดียวกันได้<br>เนื่องจากติดปัญหาเรื่องการจ่ายไฟ | ยังไม่สามารถแก้ไขได้ในปัจจุบัน<br>เนื่องจากข้อจำกัดด้านงบประมาณและความรู้ความสามารถของผู้จัดทำ<br>จึงแยกบอร์ดและแยกการจ่ายไฟแต่ละฝั่งไปก่อน |
| 2   | ไฟจ่ายไปพอ                                                                                                             | ใช้แหล่งพลังงานที่คุณภาพที่ดีขึ้น                                                                                                           |
| 3   | โครงหน้าช่วงแรก (โฟม EVA และไม้ไผ่) มีความแข็งแรงน้อยกว่าที่ต้องการและน้ำหนักเอียงไม่<br>ด้านหน้าทำให้ก้ม              | เปลี่ยนมาใช้แผ่น Plaswood ซึ่งมีความแข็งแรงมากกว่าและตัดหน้าให้สั้นลง                                                                       |

## บทที่ 5 สรุปผลและข้อเสนอแนะ

### 5.1 สรุปผล

โครงการ Proton Face สามารถพัฒนาระบบแสดงใบหน้าด้วย LED Matrix ชิป MAX7219 ได้ตามวัตถุประสงค์ โดยฝั่งขวาใช้ Arduino Nano ร่วมกับเซนเซอร์สัมผัสเปลี่ยนตา และฝั่งซ้ายใช้ ESP32 Dev V1 ซึ่งสามารถแสดงสีหน้า 8 แบบ ปรับความสว่าง กะพริบตา เปลี่ยนสีหน้าอัตโนมัติ วาดภาพเอง และเล่นเกมผ่านหน้าเว็บได้ โดยไม่ต้องใช้อินเทอร์เน็ต อย่างไรก็ตาม ทั้งสองฝั่งยังใช้บอร์ดแยกกัน ไม่ได้รวมเป็นบอร์ดเดียว เนื่องจากปัญหาการจ่ายไฟที่ยังไม่สามารถแก้ไขได้ในปัจจุบัน ซึ่งเป็นข้อจำกัดด้านงบประมาณและความรู้ความสามารถของผู้จัดทำ

### 5.2 ข้อเสนอแนะและแนวทางพัฒนาต่อ

- เพิ่มสีหน้าและแอนิเมชันปากเคลื่อนไหวตามเสียง โดยใช้ไมโครโฟน
- เพิ่มการควบคุมด้วยเซนเซอร์ เช่น ปุ่มสัมผัส หรือเซนเซอร์วัดความเร่ง
- เพิ่มระบบรหัสผ่านและความปลอดภัยของ Wi-Fi ให้รัดกุมขึ้น  
และไม่ควรเขียนรหัสผ่านไว้ในโค้ดที่เผยแพร่สาธารณะ
- ออกแบบแผงวงจร (PCB) และโครงหน้ากากให้เล็กและเบาขึ้นเพื่อสวมใส่ได้สะดวก
- ย้ายตารางสีหน้าไปเก็บเป็นไฟล์ใน SPIFFS/LittleFS เพื่อเพิ่มสีหน้าโดยไม่ต้องคอมไพล์ใหม่
- ศึกษาและแก้ปัญหการจ่ายไฟ (เช่น ออกแบบวงจรจ่ายไฟที่รองรับกระแสรวมของจอทั้งหมด)  
เพื่อรวมทั้งสองฝั่งให้ใช้บอร์ดและแหล่งจ่ายไฟร่วมกัน เมื่อมีงบประมาณและความรู้เพิ่มเติม

## บรรณานุกรม

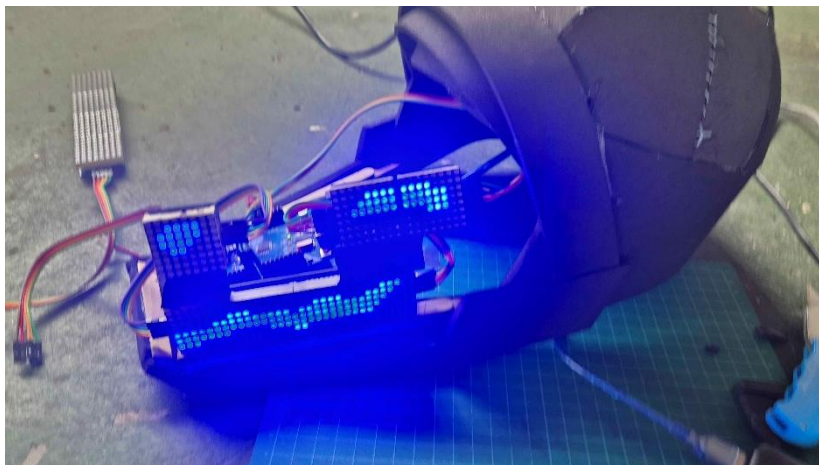
- [1] Analog Devices (Maxim Integrated). MAX7219/MAX7221 Serial Interfaced, 8-Digit LED Display Drivers (Datasheet).
- [2] Espressif Systems. ESP32 Technical Reference Manual.
- [3] Arduino. Arduino Reference. <https://www.arduino.cc/reference/>
- [4] MaxMatrix Library for Arduino. <https://github.com/AndreasBur/MaxMatrix>
- [5] ซอร์สโค้ดโครงการ: [github.com/theodor-d/Theodor-Protogen](https://github.com/theodor-d/Theodor-Protogen)

## ภาคผนวก

### ภาคผนวก ก ไฟล์ซอร์สโค้ด

| ไฟล์                              | รายละเอียด                                                           |
|-----------------------------------|----------------------------------------------------------------------|
| <a href="#">theodor_proot.ino</a> | โปรแกรมใบหน้าฝั่งขวา: Arduino Nano (MaxMatrix + เซนเซอร์สัมผัส)      |
| <a href="#">proot2_0.cpp</a>      | โปรแกรมใบหน้าฝั่งซ้าย: ESP32 Dev V1 v5.0 (Wi-Fi AP, เว็บควบคุม, เกม) |

### ภาคผนวก ข ภาพประกอบเพิ่มเติม



ภาพที่ ข.1 แรกเริ่มใช้ไมโครคอนโทรลเลอร์ support โครงสร้าง



ภาพที่ ข.2 design แรกทรงจุ่มกยาว



ภาพที่ ข.3 design หลังตัดหน้าให้สั้นลงและย้าย matrix มาไว้ด้านหน้า



ภาพที่ ข.4 design ถอด visor(ขวา) ใส่ visor(ซ้าย)